

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Отчёт по производственной преддипломной практике

Выполнил: ст. группы ПВ-223
Пахомов Владислав Андреевич

(подпись студента)

Проверил: Картамышев Сергей Владими-
рович _____
(подпись руководителя практики)

Оценка _____

Белгород 2026 г.

Оглавление

1 ВВЕДЕНИЕ

2 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ

2.1	Актуальность разрабатываемого сервиса	
2.2	Анализ существующих решений	
2.2.1	Анализ сервисов хранения аудиоданных	
2.2.2	Анализ алгоритмов хранения данных	
2.3	Процесс уменьшения размерности информации	
2.3.1	Автоэнкодер	
2.3.2	Обзор архитектуры перцептронов	
2.3.3	Обзор преобразования входных данных	
2.3.4	Функции потерь	
2.4	Постановка задачи	

1 ВВЕДЕНИЕ

Мир современного человека всё больше переходит в цифровое пространство, и это не удивительно. Цифровое пространство обладает неоспоримыми преимуществами при работе с информацией:

- Дешёвое хранение информации, а значит её большой объём
- Большая скорость передачи информации
- Возможность обрабатывать большое количество информации очень быстро
- Качество хранимой информации

Эти факторы так или иначе и стали решающими в решении хранения данных как в быту человека, так и в бизнес-процессах и в творчестве.

Самым ярким и приближённым к цифровому пространству примером является профессия программиста. На это влияет как сама специфика и предназначение профессии, так и выше-описанные процессы. Можно ли себе представить программиста, который совсем не владеет компьютером? Конечно же нет. Возьмут ли на работу программиста, который продолжает хранить весь свой код на листах бумаги? Только если эта компания почему-то любит запускать очень старые ракеты. Современного конкурентноспособного разработчика сложно представить без умных текстовых редакторов, хранения кода в облаке, автоматизации процессов доставки клиентам, нейронных сетей помогающих в написании кода (а иногда ещё и пишущих за них).

Сама профессия по личному мнению автора лежит на стыке творческого процесса и бизнес процессов в разной степени важности в зависимости от разрабатываемого продукта. Она требует как изобретательности, креативности и нестандартных подходов при решении различных задач, так и соответствия конкретным рамкам, условиям и отлаженным процессам. На примере профессии программиста можно увидеть, насколько сильно может интегрироваться цифровое пространство в сферу требующую как творческого подхода, так и точности, и насколько много пользы оно может принести при решении задач. Важно, однако, отметить, что цифровое пространство всё ещё не может полностью вытеснить другие подходы. Это такой же инструмент, как, например, молоток. Оно решает конкретные проблемы, связанные преимущественно с информацией.

Одна из сфер, которая очень похожа на программирование, является музыкальная сфера. Она также требует творческого подхода, но также задаёт в себе определённые рамки.

И да, за последнее время интеграция цифрового пространства в этой сфере тоже сильно продвинулась. Сервисы музыкального стриминга, согласно исследованиям, используют уже более трети россиян, а рост аудитории продолжается. В процессе написания музыки всё чаще используются DAW, умеющие работать с продвинутым оборудованием записи; синтез звука и его обработка тоже проходили очень долгий путь эволюции.

Однако на рынке продюсирования музыки почему-то очень мало инструментов, которые помогают в разработке аудиоданных. Множество продюсеров всё ещё складывают свои проекты в крайне неорганизованное хранилище часто представляющее собой обычную папку на локальных персональных компьютерах. А если данные будут утеряны, то обычно с этим ничего поделать нельзя и их приходится восстанавливать "по памяти".

И такие проблемы очень легко решить. Тем более, для них уже есть решение. И их можно позаимствовать из той же ранее рассмотренной профессии программиста. Облачное хранилище данных позволит избежать проблемы утери данных а также позволит делиться ими касанием пальца, что тоже повысит эффективность общения с коллегами. Хранение предыдущих версий позволит возвращаться к старым концепциям, пересматривать их, рефлексировать и адаптировать в разработке. Распространение данных в централизованной платформе гораздо более удобно для конечного потребителя в лице дистрибьютора или слушателя. Дополнение платформы в виде бизнес-сущностей позволит дистрибьюторам при публикации полученных композиций.

Создание единой платформы также создаёт множество задач по обработке и выдаче данных. Аудиоданные могут быть очень большими, особенно если это демонстрационные записи, не отличающиеся особой лаконичностью. В связи с этим должна быть также возможность порционной выдачи данных в реальном времени.

Первая глава посвящена анализу предметной области, в ней рассматривается актуальность создания сервиса, выполняется сравнительный анализ готовых решений, формируются функциональные и нефункциональные требования к разрабатываемому сервису.

Вторая глава описывает проектирование: общие архитектурные паттерны, рассмотрена клиент-серверная составляющая и интерфейсы их взаимодействия, архитектура хранилищ данных

Третья глава содержит структуру программного обеспечения: модели и сервисы, конкретные библиотеки при реализации.

Четвёртая глава описывает инфраструктуру сервиса: процессы контейнеризации, выдачи статических данных, конфигурации Nginx и обратного проксирования, подключения графических устройств для работы алгоритма сжатия.

Разрабатываемый сервис соответствует концепциям и тенденциям развития отрасли, поддерживаемый и расширяемый. Цель реализуемого сервиса - управление процессами разработки композиции а также упрощение её распространения в профессиональной среде.

2 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ

2.1 Актуальность разрабатываемого сервиса

Разработка музыкальных композиций занимает одно из важных мест на сегодняшнем рынке. Количество потребителей музыкального контента каждый день продолжает расти, о чём говорит статистика музыкальных стриминговых сервисов. Однако инструментов для управления разработки и обеспечения её эффективности у современных продюсеров представлено очень мало.

На текущий момент самым распространённым способом хранения аудиоданных является их хранение на локальных носителях или же на хранителе персональных компьютеров. Повреждение или утеря хранилищ приведёт к утере данных а также к необходимости их восстановления с нуля, что будет занимать очень много времени а также имеет риск привести к невозможности восстановления изначального варианта ввиду несовершенства человеческой памяти.

Важным аспектом является развитие композиции. Один проект или же композиция могут иметь несколько различных форм. Например, может измениться инструментальная или вокальная партия. Кроме того, отслеживание предыдущих версий позволит найти более удачный вариант исполнения и использовать наработки в новой итерации. Также вид композиции может сильно разниться в зависимости от его целей и вариантов исполнений. В отношении живой музыки это может быть композиция записанная в живом исполнении или в студии. Путём введения запрета на редактирование и удаление данных можно обеспечить как улучшенные гарантии хранения данных, так и сохранение всех версий композиции.

Для возможности отделения музыкальных композиций по их назначению можно ввести понятие тегов. Тег - короткое строковое описание версии, которое описывает цель этого релиза. При условии наличия большого количества версий композиции возникает вопрос составления серий композиций, или же плейлистов. Так как каждый из треков имеет свою определённую ценность и форму, плейлист так же может иметь свою ценность. Например, владельцу площадки для выступлений будет не очень интересно услышать, как группа будет звучать в живой записи, гораздо большую ценность для него будет предоставлять живое выступле-

ние. Плейлисты позволят быть гораздо более гибким и предоставлять различным заказчикам интересную для его задач информацию.

Предоставление служебной информации также сильно облегчит процесс взаимодействия с проектом. В любой момент можно посмотреть название проекта, исполнителей, описание без дополнительного обращения, что позволит увеличить эффективность разработки.

Важным аспектом является налаживание коммуникации между командой разработчиков. Самым простым способом может быть система комментирования, позволяющая оставить наработки, мысли и отзывы команды разработки касательно композиции.

Возможность запустить сервис на собственных серверах может стать решающей для большинства компаний для обеспечения в первую очередь конфиденциальности информации.

Введение запрета на редактирование и удаление композиций неизбежно ведёт к использованию большого объёма памяти. Необходимо выбрать алгоритм, кодек или же разработать собственный подход для хранения аудиоданных.

Таким образом, разрабатываемый сервис позволит получить инструмент, обеспечивающий централизованное защищённое хранилище наработок в музыкальной сфере, позволяющее наладить рабочие процессы внутри команды а также наладить общение с заказчиками.

2.2 Анализ существующих решений

2.2.1 Анализ сервисов хранения аудиоданных

Готовых продуктов, обеспечивающих описанную логику, очень немного, вот несколько самых популярных из них:

- Git
- Splice
- DropBox
- Boombox.io

Проведём сравнительный анализ.

Git, хорошо известный в среде программирования, не является программным обеспечением, направленным именно на хранение аудиоданных или любых других больших медиа данных.

Он лучше всего справляется именно с текстовой информацией. Кроме того, Git - это система контроля версий, а не отдельный сервис. Он хранит историю об изменениях, но не имеет привязки к какому-либо сервису. Это же и преимущество, так как декомпозиция на сервис и про-

грамму позволяет публиковать свои данные на любой совместимый продукт, вроде GitHub, GitLab, BitBucket или же развёрнутый самостоятельно сервис. Систему плейлистов можно попробовать воссоздать внутри папки, а значит по возможности распространения он сильно уступает. Git обеспечивает надёжное хранение данных, использует концепцию невозможности удаления и изменения данных, очень надёжен и имеет систему тегов. Сервисы позволяют наладить отличную работу в команде с использованием запросов на слияние, комментариев. Git и сервисы невероятно гибки, но они дают слишком много свободы и предлагают слишком мало удобств для данной задачи.

Splice на сегодняшний день отошёл от концепции хранения версионных аудиоданных и стал скорее сервисом, содержащим сэмплы - короткие музыкальные фразы. Ранее сервис предлагал облачное хранилище для проектов в DAW и коллаборативную работу над ними в облачном пространстве. Всё ещё нет системы плейлистов и обозначения композиций и нет возможности развернуть собственное хранилище.

DropBox - этот вариант чуть лучше, чем хранение на диске. Как минимум, данные уже содержатся в облаке, а значит к ним есть доступ. Существуют инструменты для интеграции с DAW. Но это нельзя назвать полноценным сервисом для хранения аудиоданных. Это просто сервис хранения данных в облаке, который через плагины используется для хранения аудио. DropBox поддерживает создание тегов и версионирования файлов, создания своих свойств для файлов. Но создания обновляемых плейлистов всё ещё нет. Эта система не заточена именно под хранение аудиоданных.

Boombox.io - самый близко подобранный к требованиям сервис. Он умеет создавать версии треков, приватные плейлисты. Композиции снабжаются метаданными а также в нём много инструментов ИИ. Существует комментирование с указанием временных меток. Однако данные изменяемые - это значит, что хранимые они могут быть нарушены. Сервис находится исключительно в облаке и не может быть предоставлен компаниям в виде сервиса для собственного развёртывания.

Таблица 1: Сравнительная таблица сервисов хранения аудиоданных

	Git	Splice	DropBox	Boombox.io
Хранение данных в облаке	Зависит от сервиса	Есть, облачный сервис	Есть, облачный сервис	Есть, облачный сервис

Продолжение таблицы 1

	Git	Splice	DropBox	Boombox.io
Версионирование и тегирование	Есть, полная система	В данный момент устарела, отсутствует	Есть 180-дневная история изменений по платной подписке	Присутствуют версии, нет тегов
Инструменты коллаборации	Зависит от сервиса (комментарии, запросы на слияние и пр.); Отдельные ветки и слияние веток	Совместная работа в DAW	Совместная работа над файлами, комментирование	Комментарии с указанием таймкодов, обычные комментарии
Поддержка разворачивания собственного сервиса	Есть	Нет, облачный сервис	Нет, облачный сервис	Нет, облачный сервис
Создание плейлистов с указанием версий	Возможно, но неудобно	Нет	Возможно, но неудобно	Есть
Указание метаданных	Выполняется вручную - неудобно	Есть	Есть, нестандартизированно	Есть, выполняется полуавтоматически
Неизменность данных	Есть	Нет	Нет	Нет

Продолжение таблицы 1

	Git	Splice	DropBox	Boombox.io
Оптимизации для хранения аудиоданных	Выполняются вручную - неудобно	Не имеет значения	Выполняется вручную	Нет

Проведённое исследование показывает, что сервисы можно разделить на две категории.

Первая категория - это сервисы предназначенные для работы исключительно с одной конкретной задачей. Git решает вопрос версионирования, а DropBox решает проблему хранения в облаке. Обычно работа с такими инструментами требует дополнительной обработки, оптимизации и введения правил, по которым они будут храниться. Они предоставляют решение одной конкретной задачи и потому слишком универсальны, они не предоставляют оптимизаций для работы конкретно с аудио данными.

Вторая категория - сервисы, направленные непосредственно на работу с аудио. Начнём с того, что они крайне непопулярны. Их очень мало, и поэтому, чтобы продолжать выживать, они инкапсулируют свои собственные решения на своих серверах и предлагают их за плату. Такие решения могут быть угрозой конфиденциальности для бизнеса.

Моё решение будет брать лучшее из двух категорий. В нём можно найти специфичный для аудиоданных и их распространения функционал, а также решённые задачи версионирования и хранения данных в облаке. Кроме того, текущее решение распространяется в свободном доступе.

2.2.2 Анализ алгоритмов хранения данных

Алгоритмы кодирования аудиоданных прошли очень долгий путь развития, рассмотрим несколько из них

- MP3
- Opus
- FLAC
- WAV

MP3 - аудиокодек с потерей данных. Этот кодек использует психоакустические оптимизации для того, чтобы сократить количество занимаемого места. Из-за этих оптимизаций, особенно

на маленьком битрейте, может возникать очень множество артефактов при прослушивании. Звук становится наиболее чистым и неотличимым от исходного при 320 КБит/с. Однако, даже при большом битрейте, кодек всё ещё допускает потери данных, что недопустимо в профессиональной сфере.

Opus - современный кодек, используемый в основном для стриминга ввиду его очень маленького битрейта (около 96 КБит/с) при котором он становится неотличим от оригинального звука. Недостатком является то, что всё ещё очень мало устройств могут поддерживать этот формат, а также он даёт большую нагрузку для устройства воспроизводящего аудио. Кроме того, кодек тоже допускает потерю данных, что может быть заметно на хорошем оборудовании. Кодек Opus оптимизирован под одну конкретную частоту дискретизации - 48000 Гц, что так же ограничивает его использование в студиях.

FLAC - старый формат без потери данных. Он использует модифицированный алгоритм кодирования Хаффмана - кодирование сдвига Хаффмана. В фрейм записываются не повторяющиеся данные, а их линейный рост. Такой метод сжатия позволяет достичь коэффициента сжатия в 2 раза, что меньше чем предыдущие кодеки, однако и потери данных в нём нет.

WAV - кодек, содержащий необработанный уровень амплитуды без сжатия. Практически не влияет на загруженность устройства, однако и содержит в себе больше всего данных. Идеально подходит для записи и использования в DAW ввиду простоты его распаковки, но плох для длительного хранения.

Таблица 2: Сравнительная таблица кодеков

	MP3	Opus	FLAC	WAV
Коэффициент сжатия	10	17	1.7	Без сжатия
Нагрузка при кодировании/декодировании	Средняя	Высокая	Низкая	Крайне низкая

Продолжение таблицы 2

	MP3	Opus	FLAC	WAV
Поддержка кодеков на устройствах	Очень хорошая поддержка на старых и новых устройствах	Работает на- тивно только на относи- тельно новых устройствах	Хорошая поддержка на старых и новых устройствах	Может плохо работать на операцион- ных систе- мах кроме Windows, в остальном довольно хорошая поддержка
В восста- новленных данных нет потерь	Нет	Нет	Да	Да
Использование в стриминге	Хорошо под- ходит для стриминга	Лучше всего подходит для стриминга	Редко ис- пользуется в стриминге, только там где важно качество	Требуется очень боль- шого про- пускного канала для стриминга
Битрейт	320 КБит/с	96 КБит/с	800 КБит/с	1411 КБит/с
Влияние на исходный материал	Сильное ис- пользование психоаку- стических принципов	Сильное ис- пользование психоаку- стических принципов	Нет	Нет

Кодеки MP3 и Opus подходят для хранения аудиоданных для конечного потребителя, так как включают в себя психоакустические приёмы: затухание, удаление неслышимых данных, маскирование. При дополнительной обработке данных в DAW утерянные данные могут повлиять на результат выполняемой работы, поэтому для текущей задачи эти кодеки использовать можно, но только в определённых сценариях, вроде подготовки выпуска композиции

в финальных сервисах дистрибуции.

Кодеки FLAC и WAV же не искажают исходные данные, а значит они лучше подходят для композиций записанных в данном формате. Но их узким местом является количество занимаемых данных. FLAC частично решает эту проблему используя математические методы кодирования вместо психоакустических. Однако степень сжатия данных в особенно сложных местах может быть достаточно маленькой.

Можно модифицировать алгоритм FLAC и предоставить возможность кодировать особо сложные фреймы при помощи методов, допускающих потерю данных, однако не влияющих на исходную волну амплитуды достаточно сильно.

2.3 Процесс уменьшения размерности информации

Процессы уменьшения размерности информации это задачи которые позволяют сократить количество свойств внутри какого-либо объекта за счёт нахождения связи между свойствами этого объекта.

Существует множество способов, позволяющих сократить размерность данных. Математические способы обычно используют матрицу корреляции для определения взаимосвязи между свойствами внутри объекта. По этой же матрице данные можно восстановить обратно.

Однако аудиоданные часто могут содержать более комплексные связи, которые невозможно выразить при помощи таких матриц. В небольшом отрезке из аудиофайла могут быть как сильные всплески амплитуды, так и постоянные данные. Однако музыкальная композиция всё ещё подчиняется гармоническим частотным правилам, соотношениям частот внутри аккордов, темпу и ритму.

Для нахождения неочевидных и сложных закономерностей всё чаще применяются свёрточные нейросети (CNN).

2.3.1 Автоэнкодер

Автоэнкодер по своей структуре очень похож на обычную свёрточную нейросеть. Он так же состоит из перцептронов, у него есть скрытые слои, входной и выходной слои. Однако есть в архитектуре и различия.

Автоэнкодер можно разделить на две составляющие: энкодер и декодер. Задача энкодера - уменьшить размерность данных. Каждый из последующих слоёв уменьшает размерность данных. Получившийся вектор называют латентным вектором или же латентом. Он содержит в себе только самую важную информацию. Входным вектором для декодера как раз

будет являться латентный вектор. Задача декодера - восстановить пробелы в латенте чтобы получить исходный объект. Если энкодер шёл на уменьшение количества свойств объекта, то декодер уже увеличивает их количество в обратном порядке.

Чем меньше разрыв между количеством перцептронов в каждом слое, тем более качественным должен получиться результат упаковки/распаковки. Однако с увеличением количества перцептронов нейросеть будет обучаться гораздо дольше, проход будет выполняться дольше а ресурсов обеспечивающих обучение и использование нейросети может не хватить. На рисунке 1 изображен пример структуры автоэнкодера.

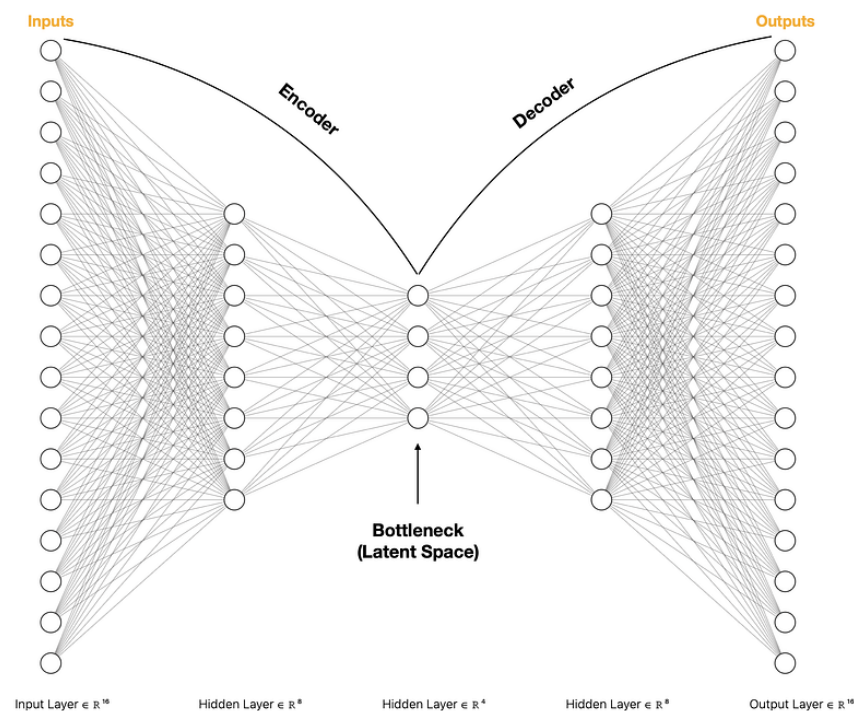


Рисунок 1: Пример структуры автоэнкодера [1]

Автоэнкодеры находят своё применение не только в задачах уменьшения размерности вектора, но и в задачах преобразования исходных данных. Например, удаление шума с изображения, как показано на рисунке 2.

У данной архитектуры нейронной сети для текущей задачи есть как положительные, так и отрицательные стороны. Часто для восстановления музыкального отрезка очень важен контекст: что звучало до текущего отрезка, так как контекст содержит информацию о волнах. Некоторые особенно низкие частоты могут быть и не определены как частота в виду достаточно маленького времени. Минимально слышимая для человеческого уха частота в 20 Гц потребует около 0,05 секунд для проигрывания одного периода. Проблему контекста в данной архитектуре можно решить при помощи увеличения фрейма, например, до 1 секунды. Этой информации будет достаточно для сохранения частот. Однако с увеличением входного

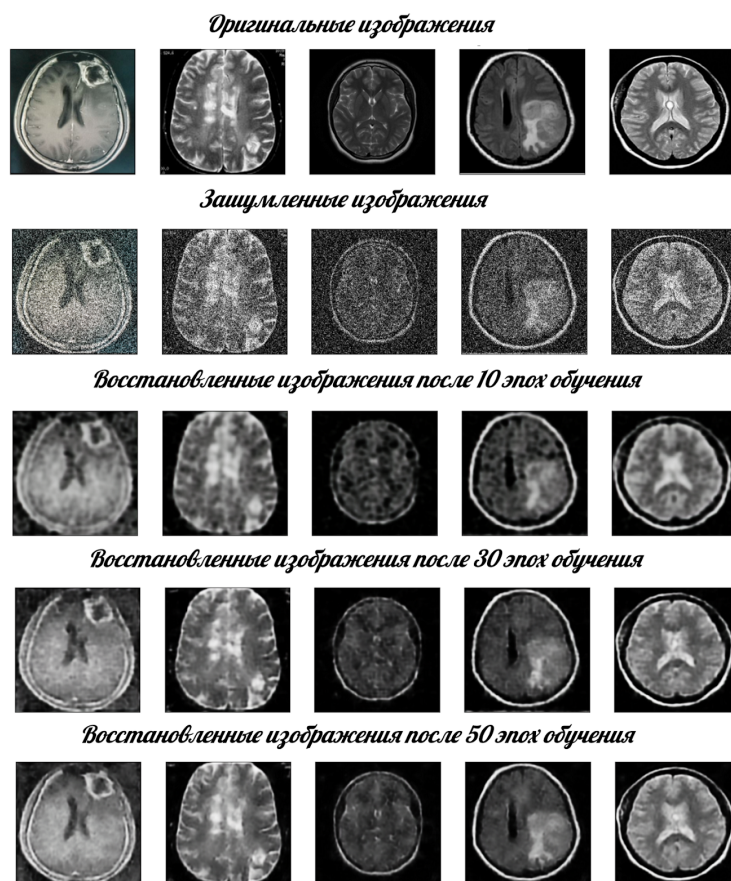


Рисунок 2: Удаление шума при помощи автоэнкодера [2]

сложность обучения, требования к аппаратному обеспечению растут. Есть и плюсы в использовании текущего подхода. Декодирование отдельных независимых друг от друга фреймов позволит масштабировать процесс декодирования при появлении более продвинутого аппаратного обеспечения путём параллельного декодирования. Кроме того, автоэнкодер довольно просто обучать и использовать.

2.3.2 Обзор архитектуры перцептронов

Как уже было сказано ранее, автоэнкодер может быть достаточно тяжёлой свёрточной нейросетью при использовании большого количества входных параметров. Существуют способы оптимизации, позволяющие сократить количество связей между нейронами.

Полносвязная нейросеть

Полносвязный слой строится по принципу "нейрон связан с каждым нейроном из предыдущего слоя". В контексте аудиоданных это может быть как положительным примером, так и отрицательным. Так, например, более низкие частоты с большим периодом смогут влиять на значение текущего нейрона, а значит информация о низких частотах будет сохранена. Но полносвязные слои крайне неоптимизированны. Вместе с важными данными в нейрон

может приходить и ”мусор который будет только добавлять шума в декодированной записи. При использовании полносвязной нейросети требуется тонкая настройка входного слоя. Далеко не все данные могут быть полезными, и слишком большой входной вектор может привести к очень большому времени достижения экстремума для нейронной сети при обучении.

Использование Convolution слоёв

Convolutional слои позволяют создавать неполносвязную нейронную сеть. На примере одномерного массива Conv-слой сможет выбрать лишь небольшой участок из более большого вектора. Кроме того, Convolution слои могут работать как с одномерными, так и с двумерными, трёхмерными слоями. Например, на рисунке 3 изображён пример поведения Convolutional слоя. Он не использует при вычислении все входные параметры, а выбирает только соответственно своему фильтру

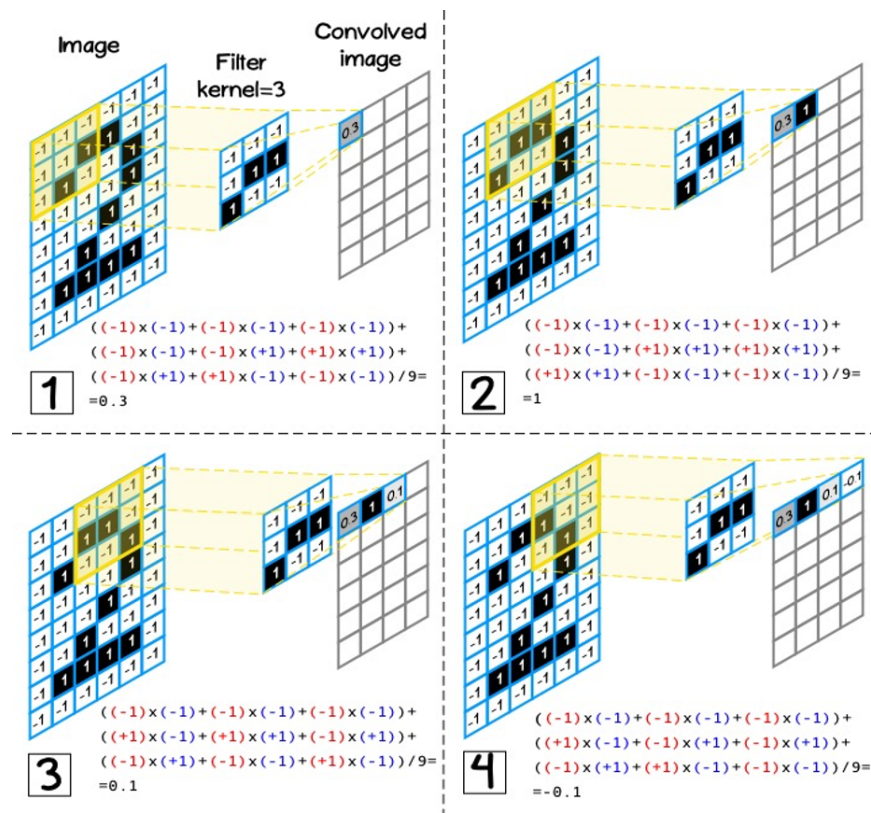


Рисунок 3: Принцип работы Convolutional Layer [3]

Использование Convolutional слоёв позволяет решить проблему слишком большой выборки данных, что позволит уменьшить количество связей между нейронами. Уменьшение связей между нейронами позволит увеличить размер входного слоя, что позволит увеличить количество кодируемой/декодируемой информации за один раз. Однако при использовании такого рода слоёв всё ещё остаётся проблема настройки выборки. Кроме того, при потоковой распаковке аудио гораздо более ценна работа с относительно небольшим фреймом данных.

2.3.3 Обзор преобразования входных данных

Нейронные сети умеют работать с данными различных размерностей. Рассмотрим, в какие форматы данных можно было бы преобразовать аудиоданные.

Одномерный вектор

Это самый простой способ. Аудиоданные преобразуются в одномерный массив, свойство каждого из которых содержит амплитуду. Этот способ относительно лёгок в реализации и не требует больших преобразовательных мощностей, так как фактически выполняется преобразование аудиоданных в последовательность амплитуд, что естественно для большинства аудиоформатов на сегодняшний день, и производится дополнительная нормализация данных для работы в нейронной сети.

Двухмерная спектрограмма

Можно преобразовать аудиоданные в более сложный и требующий больших вычислений формат. Спектрограмма представляет собой изображение. Одна из координат может представлять собой время, а другая - частоту. Яркость точки на изображении в точке (x, y) будет показывать, насколько сильна амплитуда частоты x в момент y . Для получения такого изображения используются алгоритм быстрого преобразования Фурье. Для этого способа так же требуется достаточно большой размер фрейма. Если фрейм будет недостаточно большим, то алгоритм просто "не услышит" длинные волны. На рисунке 4 представлена спектрограмма записи голоса человека, говорящего Hello World, после чего он кашляет. Видно по первому всплеску, что у голоса есть определённый тон и повторяющиеся гребни - октавы. Кашель же - это в основном шум, поэтому основной тон там не так заметен.

Использование такого подхода так же будет вызывать трудности и с декодированием спектрограммы. Для этого тоже используются специальные алгоритмы Гриффина-Лима или обратного быстрого преобразования Фурье. Данный способ даёт большую нагрузку на аппаратное обеспечение и может вызывать задержки при потоковой распаковке.

2.3.4 Функции потерь

Функции потерь будут прямо влиять на процесс обучения и, соответственно, на получившееся звучание аудиоданных. Именно функция потерь определяет то, на что будет сфокусировано внимание при обучении.

MSELoss

Самая простая функция. Предположим, что у нас есть исходный сигнал y и предсказанный сигнал $\bar{y}$ размерность N . Тогда функция потерь MSE будет высчитана как

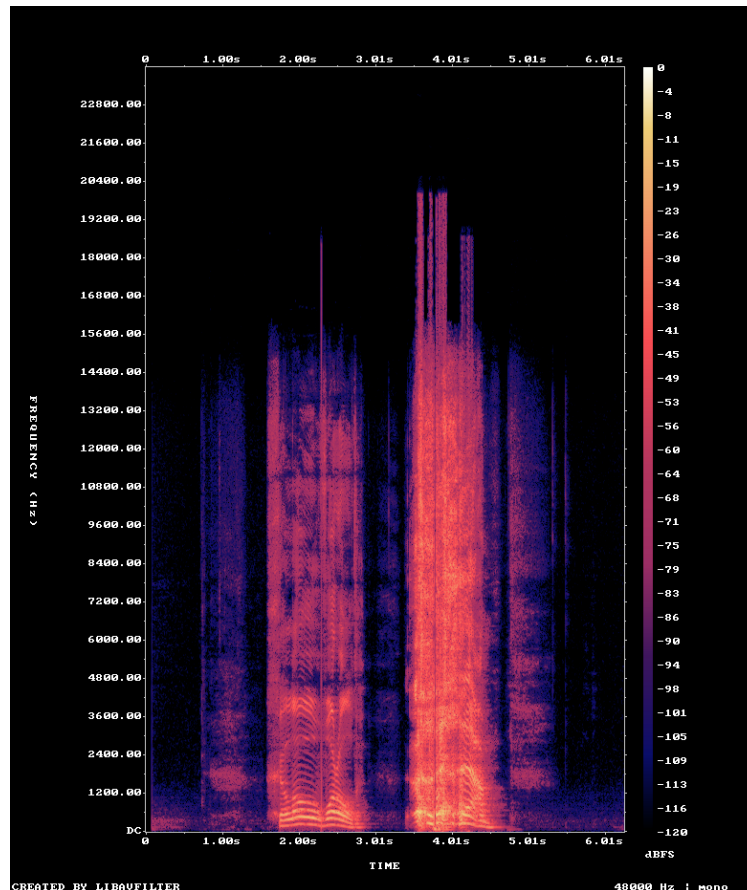


Рисунок 4: Запись голоса человека в спектрограмме, авторский материал

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \bar{y}_i)^2$$

Эта функция может быть не очень хороша для аудиоданных, так как она слабо учитывает отклонения во всплесках - в аудио это может быть удар бочки или же по тарелке. Но она неплохо восстанавливает общую картинку и общую слышимость.

Spectral Loss

Функция спектральной потери считает потери по определённым частотам. Эта функция потери гораздо более близка к природе аудиоданных, так как работает с её частотами напрямую. Обычно высчитывается потеря по нескольким частотам отражающим одну ноту, например [128 Гц, 256 Гц, 512 Гц, 1024 Гц]. Для определения амплитуд частот используется быстрое преобразование Фурье. Эта функция потерь гораздо лучше сохраняет детали, нежели общее звучание. Семейство функций спектральной потери часто используется при обучении нейронных сетей, работающих с аудиоданными.

Положим, что у нас есть исходный сигнал y и $\bar{y}$. Тогда функция спектральной потери будет высчитана следующим образом

$$SpectralLoss = |\log(|STFT(y)| + \epsilon) - \log(|STFT(\bar{y})| + \epsilon)|$$

Где ϵ - очень маленькая величина.

Чаще всего используется комбинированная функция потери в зависимости от цели при обучении.

2.4 Постановка задачи

На основе представленной информации можем сформировать список желаемого функционала в сервисе

- Аутентификация и регистрация для управления бизнес сущностями.
- Возможность работы с авторами композиции.
 - Автор должен содержать имя, аватар, описание
 - Должны поддерживаться операции CRUD: добавление, удаление, редактирование и получение
 - Необходимо предоставить графический интерфейс для работы с указанными операциями
- Возможность работы с группами
 - Группа - это объединение авторов. Она должна включать в себя аватар, описание, участников
 - Должны поддерживаться операции CRUD: добавление, удаление, редактирование и получение
 - Необходимо предоставить графический интерфейс для работы с указанными операциями
- Возможность создавать песню
 - Песня должна содержать в себе справочную информацию: имя, описание, тэг, BPM, тональность, слова, авторов и группы а также файлы этой песни.
 - Должна быть возможность указать ведущих файлов. Эти файлы репрезентируют песню. Так, ведущее изображение является картинкой этой песни. А ведущее аудио - проигрывающей песней при её запуске. Дополнительные файлы могут содержать прочую важную информацию.
 - После создания песню удалить нельзя - можно только выпустить следующий релиз песни. Всегда можно просмотреть предыдущие релизы песен.

- Должен быть доступен чат внутри каждой из песен, где пользователи могут просматривать а также отправлять свои собственные сообщения
- Должна быть реализована возможность просмотра конкретной версии песни.
- Должны быть реализованы операции в API для выполнения вышеобозначенных операций.
- Необходимо предоставить графический интерфейс для работы с указанными операциями.

- Плейлист

- Плейлист - это сущность, которая собирает в себе песни и фильтрует их по фильтру. Плейлист должен в себя включать изображение, название и описание.
- Функция фильтрации заключается в поиске по текстовому полю, который может представлять собой либо название тега либо название версии. Если указано название тега, в песне должна быть найдена наиболее новая песня с указанным тегом. Если фильтр не указан, будет предоставлена последняя версия песни. Если же указан номер версии, будет найдена версия с этим конкретным номером.
- Должны быть реализованы операции CRUD: добавление, удаление, редактирование и получение
- Должен быть предоставлен графический интерфейс для взаимодействия с указанными операциями.

- Взаимодействие с файлами

- Должен быть предоставлен альтернативный способ хранения аудиоданных, использующий доверенный кодек для хранения аудиоданных.
- Необходимо предоставить возможность скачивать загруженные файлы
- Отправка аудиоданных со своим кодеком должна поддерживать специальные заголовки, позволяющие получать данные порционно. Распаковка не должна происходить одномоментно и должна распаковываться постепенно.

- Проигрыватель

- Должен быть реализован проигрыватель, позволяющий проиграть плейлист или же отдельную версию песни.

- Плейлист должен поддерживать следующие операции: выбрать порядок в пределах плейлиста (перемешать, в обычном порядке, зациклить), выбрать порядок в передлах песни (зациклить песню или разрешить переход к следующим), выбрать следующий трек, выбрать предыдущий трек, отразить информацию о песне, выбрать громкость.
- Должно быть реализовано API достаточное для выполнения описанных функций
- Должен быть реализован графический интерфейс, поддерживающий эти операции